

Adapting Protein Language Models for Generative C05 CDR-H3 Design

Semester Project Report

Giovanni Guidarini
Semester Project Author
Student No. 25-945-684

Marco De Negri
Project Collaborator

Under the supervision of Prof. Dr. Andreas Krause and Dr. Scott Sussex

Spring 2026 | Learning and Adaptive Systems | ETH Zürich

Abstract

In this work we fine-tune a Protein Language Model to obtain a generative model with the aim of proposing novel and high-affinity variants of a wild-type antibody sequence for experimental validation. Two methods are employed: domain-adaptive pretraining (DAP) of the model on antibody sequences from the Observed Antibody Space (OAS), and fine-tuning using Direct-Preference Optimization (DPO) on a dataset of C05 variants with measured binding enrichment scores. These two methods are then combined to obtain a model that is both knowledgeable about the antibody space and biased towards generating sequences with high binding enrichment scores. We then leverage the generative capabilities of the model to propose novel variants of the C05 antibody that can be tested in a wet lab or serve as a starting point for further optimization of the antibody sequence.

1 Introduction

The goal of this project is to adapt a pretrained Protein Language Model into a generative model for the design of new variants of the C05 antibody CDR-H3 loop. We start from ESM2, which has been pretrained on a broad protein sequence distribution and therefore contains general information about protein sequence structure, but has not been specifically trained to model antibodies or to prefer sequences with high C05 binding enrichment.

We address these two limitations in two successive adaptation stages. First, we perform domain-adaptive pretraining (DAP) on antibody sequences from the Observed Antibody Space (OAS). This stage is still self-supervised: no binding measurements are used, and the objective remains masked language modeling. Its purpose is therefore to move the model from a general protein distribution toward the antibody sequence distribution before introducing task-specific supervision. Related strategies have previously been used in protein engineering by continuing unsupervised training on evolutionarily related sequences [1], while antibody-specific language models such as AbLang have shown the benefit of training directly on large antibody repertoires [2].

In the second stage, we apply Direct Preference Optimization (DPO) using deep mutational scanning (DMS) measurements of C05 variants. The DMS data provide the task-specific information that is absent from the OAS corpus: pairs of CDR-H3 variants can be ordered according to their measured binding enrichment, allowing the model to be explicitly biased toward sequences with better experimental fitness. DPO has recently been adapted to protein design, where preference optimization has been used to align protein generative models with experimentally measured fitness [3, 4].

This coarse-to-specific training strategy is conceptually similar to the staged training used for modern language models, where broad pretraining is followed by progressively more task- or preference-specific adaptation [5]. In our case, ESM2 provides general protein knowledge, DAP adds antibody-domain information, and DPO finally introduces C05-specific experimental preferences.

We evaluate whether each adaptation stage improves language-model fit and agreement with held-out DMS measurements, compare full-parameter and LoRA-based DPO, and test whether DAP provides a better initialization for preference optimization. Finally, we use the adapted models for sequence generation with Gibbs sampling and stochastic beam search, and compare the generated libraries with

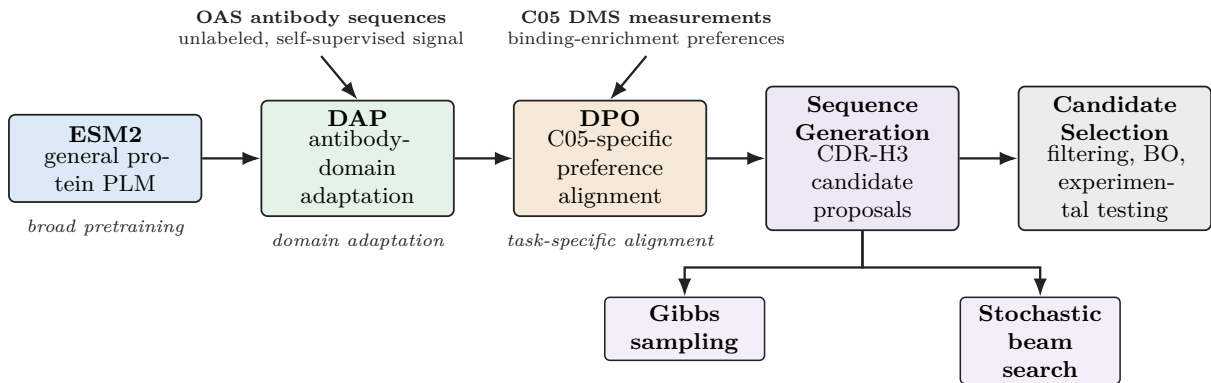


Figure 1: High-level overview of the training and generation pipeline. ESM2 provides a general protein-language-model initialization. Domain-adaptive pretraining on OAS moves the model toward the antibody sequence distribution, while DPO introduces C05-specific preferences derived from deep mutational scanning. The adapted models are then used to generate CDR-H3 variants through Gibbs sampling or stochastic beam search, producing candidate sequences for downstream filtering, Bayesian optimization, and experimental validation.

random and PSSM-based baselines in terms of novelty, diversity, predicted quality, and agreement with independent in-silico scorers. The schematic overview of the training and generation pipeline is shown in Figure 1.

The final objective is not to use the language model as a direct replacement for experimental measurement, but as a proposal mechanism that can generate candidate C05 variants for independent screening, experimental testing, or Bayesian optimization.

1.1 Related Work

Protein language models have been used as general-purpose representations for protein engineering, but several works have shown that additional adaptation to a more specific sequence distribution can improve their usefulness for downstream design tasks. Alley et al. [1] introduced UniRep and showed that continuing unsupervised training on sequences evolutionarily related to a target protein, a procedure referred to as *evotuning*, could improve performance on protein engineering problems. This is closely related to the first stage of our pipeline, although in our case the adaptation is performed at the antibody-domain level using sequences from OAS rather than on homologs of a single target protein. Antibody-specific language models provide further motivation for this type of adaptation. AbLang [2], for example, was trained directly on antibody sequences from OAS using a masked-language-model objective, showing that antibody repertoires contain domain-specific sequence information that can be learned by dedicated language models.

A separate line of work has investigated how pretrained protein generative models can be aligned with experimentally measured properties. ProteinDPO [3] adapts Direct Preference Optimization to protein design by constructing preferences from experimental fitness measurements and using them to align a pretrained structure-conditioned Protein Language Model. More recently, g-DPO [4] proposed a more scalable formulation of preference optimization for protein language models by reducing redundant pairwise comparisons and approximating likelihood computations. These works motivate our use of DPO to incorporate C05 binding-enrichment measurements into ESM2. In contrast to ProteinDPO, which focuses primarily on protein stability and uses a structure-conditioned autoregressive model, our setting uses a masked Protein Language Model and preference data from a deep mutational scan of a single antibody CDR-H3 loop.

The combination of domain adaptation followed by preference-based alignment also follows a more general pattern used in modern language-model training. Large language models are first pretrained on broad unlabeled corpora and are subsequently adapted using increasingly task-specific supervision. For example, InstructGPT [5] starts from a pretrained language model and applies supervised fine-tuning followed by preference-based reinforcement learning using human feedback. Our setting is different in both model architecture and supervision, but follows the same high-level progression from broad unsupervised pretraining to domain-specific adaptation and finally to preference-based task alignment. In our pipeline, ESM2 provides general protein knowledge, DAP introduces antibody-domain information, and DPO

incorporates C05-specific experimental fitness preferences.

2 Background

2.1 Protein Language Models and ESM2

Protein Language Models (PLMs) are a category of machine learning models which, as the name suggests, apply principles and architectures from Natural Language Processing to the domain of protein sequences. The main idea behind this kind of tools is that the statistical properties of protein sequences reflect underlying chemical and physical properties of the proteins themselves, and thus a model that is trained on a large corpus of protein sequences can develop an internal representation of the proteins that encodes information about their structure and function.

More specifically, PLMs are transformer-based models trained on large datasets of protein sequences using self-supervised learning techniques. The most common self-supervised learning technique used for PLMs is Masked Language Modeling (MLM), where the model is trained to predict missing amino acids in a sequence given the surrounding context.

For our project we used the ESM 2 model [6], an open-source PLM developed by Meta AI, which is trained on the UniRef50 dataset, a large collection of protein sequences from the UniProt database. Although not being the most recent or the most performant PLM, ESM 2 is a widely used and well-documented model, which makes it suitable for our project, as it is a well-established baseline for protein design tasks. ESM 2 is available in different sizes, from 8 million parameters to 15 billion parameters, for our project we used several sizes of the model: 35 million, 150 million and 650 million parameters, according to the needs of each experiment. In particular, the 650 million parameter model was used for the DAP and DPO experiments, while the lightweight 35 million parameter model was used for investigating whether a DAP-adapted model is more data efficient than the base model.

2.2 Domain-Adaptive Pretraining

Domain-adaptive pretraining (DAP) is a second phase of self-supervised pretraining on unlabeled data from the target domain [7]. Here, continued masked language model training on OAS antibody sequences adapts ESM2 from its broad protein pretraining distribution to the antibody domain.

2.3 Direct Preference Optimization

Direct-Preference Optimization [8] is a technique invented for fine-tuning large language models. This technique can be applied with few adaptations to Masked Protein Language Models such as ESM 2 [3]. The idea behind this technique is simple: we first draw pairs of samples from our datasets, and label the one with the most desirable properties as the winner (w) and the other one as loser (l). Then we train the model using a specific loss function that encourages the model to prefer (i.e. assign higher probability to) the winner sequence of the pair rather than the loser. The dataset thus consists of pairs of sequences, for our purpose the discriminating factor that determines whether a sequence is the winner or the loser in the pair is its binding enrichment score: the sequence with the best score is labeled as winner and the other one as loser:

$$\mathcal{D} = \{(y^w, y^l) : \text{binding_enrichment}(y^w) > \text{binding_enrichment}(y^l)\} \quad (1)$$

The derivation of such loss function has its roots in Reinforcement Learning, but the loss itself can be understood even without having to rely on this machinery (for a more detailed explanation, see [9, 8])

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(y^w, y^l) \sim \mathcal{D}} \left[\log \sigma \left(\beta \log \frac{\pi_{\theta}(y^w)}{\pi_{\text{ref}}(y^w)} - \beta \log \frac{\pi_{\theta}(y^l)}{\pi_{\text{ref}}(y^l)} \right) \right] \quad (2)$$

Following [3], we use a weighted variant of the DPO loss function; for a preference pair (y^w, y^l) , the weights are defined as

$$w^{(i)} = \frac{\exp \left(\beta \log \frac{\pi_{\theta}(y^{(i)})}{\pi_{\text{ref}}(y^{(i)})} \right)}{\sum_{i \in \{w, l\}} \exp \left(\beta \log \frac{\pi_{\theta}(y^{(i)})}{\pi_{\text{ref}}(y^{(i)})} \right)} \quad \text{for } i \in \{w, l\}$$

The weighted DPO loss function is then

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(y^w, y^l) \sim \mathcal{D}} \left[w^w \log \sigma \left(\beta \log \frac{\pi_{\theta}(y^w)}{\pi_{\text{ref}}(y^w)} - \beta \log \frac{\pi_{\theta}(y^l)}{\pi_{\text{ref}}(y^l)} \right) + w^l \log \sigma \left(\beta \log \frac{\pi_{\theta}(y^l)}{\pi_{\text{ref}}(y^l)} - \beta \log \frac{\pi_{\theta}(y^w)}{\pi_{\text{ref}}(y^w)} \right) \right]$$

This is the weighted preference objective used in our experiments.

By definition, the weights w^w and w^l define a probability distribution over the winner and loser sequences, hence the weighted DPO loss function can be interpreted as a weighted average of the contributions of the winner and loser sequences, where the weights are determined by the relative probabilities assigned by the model to each sequence.

2.4 Low-Rank Adaptation

Low-Rank Adaptation (LoRA) [10] is a Parameter-Efficient Fine-Tuning (PEFT) technique that allows to fine-tune large models by updating only a small number of parameters, hence providing a more efficient and scalable approach to model adaptation. The main assumption behind LoRA is that the update to the weights of a pre-trained model can be approximated by a low-rank decomposition, which allows to reduce the number of trainable parameters. In particular, let $W_0 \in \mathbb{R}^{d \times k}$ be the weight matrix of a pre-trained model, we can express the update to this weight matrix as a low-rank decomposition:

$$W = W_0 + \Delta W = W_0 + \frac{\alpha}{r} BA$$

where $A \in \mathbb{R}^{r \times k}$ and $B \in \mathbb{R}^{d \times r}$ are low-rank matrices with rank $r \ll \min(d, k)$. In this way, instead of updating the full weight matrix W_0 , only the low-rank matrices A and B are updated during fine-tuning, which significantly reduces the number of trainable parameters. In the context of our project, we used LoRA to fine-tune the ESM 2 model using DPO, applying the low-rank decomposition to the attention layers of the model, which are the most parameter-heavy layers of the model, while keeping the rest of the model frozen.

3 Data

3.1 The C05 Antibody and the CDR-H3 loop

Our project is focused on the C05 antibody, which is a broadly neutralizing antibody that targets the hemagglutinin (HA) protein of influenza viruses; specifically, C05 was originally isolated from human immune repertoires [11]. Its peculiarity is that it binds to the receptor-binding site (RBS) of the HA protein, which is a highly conserved region across different influenza strains, making the C05 antibody capable of neutralizing diverse influenza viruses.

An important feature of the C05 antibody is that its interaction with the HA protein is mediated essentially by its complementarity-determining region 3 of the heavy chain (CDR-H3) loop, while the light chain of the antibody does not contribute significantly to the binding. The CDR-H3 loop is a hypervariable region of the antibody that is responsible for recognizing and binding to antigens. The CDR-H3 loop of the C05 antibody is composed of 24 amino acids, an exceptionally long length for a CDR-H3 loop, which is usually between 8 and 16 amino acids long. In our project we focused on the CDR-H3 loop of the C05 antibody, in particular we aim to generate novel variants of the CDR-H3 loop that can bind to the HA protein of influenza viruses with higher affinity than the wild-type sequence. C05 binding is dominated by the heavy-chain CDR-H3 interaction with the HA receptor-binding site, motivating our decision to restrict design to the CDR-H3 loop.

3.2 Deep Mutational Scanning Datasets

The datasets that were used for our project are the result of a deep mutational scanning experiment [12] (DMS), which is a high-throughput experimental technique that allows to measure the effects of a large number of mutations on the function of a protein. At our disposal we have multiple datasets that differ in the number of mutations and in the influenza strain that was used for the binding assay. In particular we have three binding enrichment datasets per strain: a dataset with nominal edit distance 2

(ED2), one with edit distance 5 (ED5) and one with edit distance 8 to 11 (ED8-11). The datasets with edit distance greater or equal to 5 showcase mutations only in fixed positions of the CDR-H3 loop, these positions were selected based on the results of the ED2 dataset as the ones in which mutations were the most beneficial. In all datasets the number of mutations is not fixed, due to noise in sequencing and synthesis, the sequences in the datasets can have less or more mutations than the nominal edit distance. Each dataset contains the amino acid sequence of each variant, the count of the times that each sequence bound and did not bind to the target, and the (adjusted) binding enrichment score, which is computed as:

$$\text{binding_enrichment} = \log \left(\frac{\text{binding_count} + 0.5}{\text{non_binding_count} + 0.5} \right) + \log \left(\frac{N}{B} \right) \quad (3)$$

where B is the total number of binding counts in the dataset and N is the total number of non-binding counts in the dataset. We chose to use the binding enrichment score as the target for our project. Moreover, we only trained our models on the M22 datasets, as it was the influenza strain with the largest data available. The distribution of the binding enrichment scores for the three datasets is shown in Figure 2.

The wild type sequence (WT) is the one with zero mutations. As showcased in Figure 2, the distribution of the binding enrichment scores is challenging for two reasons:

1. The distribution is strongly bimodal, especially for the ED5 and ED811 datasets, it has a large peak at binding enrichment around binding enrichment -5 and one large peak around binding enrichment 5. These two peaks are due to the fact that often sequences do not bind at all or do not have non-binding counts.
2. In the ED2 dataset, there are not many sequences with scores that are higher than the wild-type in terms of binding enrichment, this is expected, since it is more likely that a mutation will damage the structure and the function of the antibody, however it makes more challenging to create a model that generates sequences that have some genuine probability of outperforming the wild type.

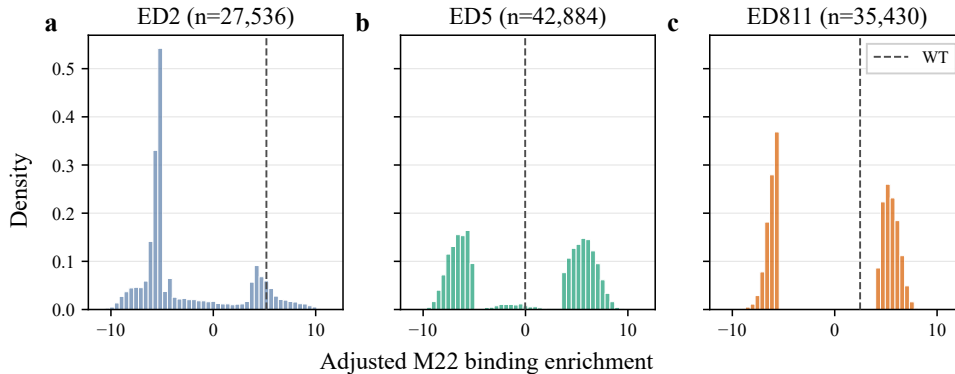


Figure 2: Binding enrichment distribution for the three M22 datasets.

The three datasets were split into train, validation and test sets, with proportions of 80%, 10% and 10% respectively. The splits were stratified by binding enrichment score, to ensure that the distribution of the scores was similar across the three splits.

3.3 Domain-Adaptive Pretraining Data

The DAP corpus was derived from the Observed Antibody Space (OAS) [13]. The OAS query was restricted to human heavy-chain sequences. Sequences were then filtered by removing exact duplicates, entries with the ANARCI status “Missing Conserved Cysteine,” and heavily fragmented sequences lacking more than 16 residues at the N-terminus or more than 7 residues at the C-terminus. Non-standard amino acids were represented by an X token. Before clustering, the filtered corpus contained 366,792,632 sequences. Redundancy was further reduced with Linclust at 95% sequence identity, retaining one representative per cluster. The resulting representatives were assigned to train, validation, and test splits in proportions of 90%, 5%, and 5%, respectively, using a deterministic assignment based on sequence identifiers.

3.4 Preference Pair Construction

As the distribution of the binding enrichment scores is strongly bimodal, some attention must be paid to the construction of the preference pairs for DPO training. In particular, we require the resulting set of preference pairs to carry a sufficiently strong supervision signal, meaning that the preferred sequence has a binding enrichment score that is appreciably higher than that of the rejected sequence. At the same time, the pair construction should encourage the model to assign higher probability to sequences that outperform the wild type.

Training was restricted to the ED2 dataset, while the ED5 and ED811 datasets were retained for evaluation. Among the available DMS datasets, ED2 exhibits the broadest range of binding enrichment values and was therefore considered the most suitable source of preference supervision.

Preference pairs were derived directly from the scalar binding enrichment measurements. For each sequence, the enrichment score was expressed relative to the wild type,

$$\Delta = s - s_{\text{WT}},$$

where s denotes the measured binding enrichment score and s_{WT} the corresponding wild-type value. The wild type therefore has $\Delta = 0$. Given two sequences x_i and x_j , the sequence with the larger value of Δ is treated as the preferred sequence whenever the score difference is sufficiently large. Pairs with too small a separation in enrichment are discarded through a minimum-margin criterion.

To avoid constructing a training set dominated by a single region of the enrichment distribution, preference pairs are sampled from four distinct categories. First, *within-positive* pairs are formed between two sequences with $\Delta > 0$, with the higher-scoring sequence taken as preferred. Second, *within-negative* pairs compare two sequences with $\Delta < 0$, so that the less deleterious variant is preferred. Third, *wild-type anchor* pairs compare strongly beneficial variants against the wild type and the wild type against strongly deleterious variants. Finally, *cross* pairs compare positive and negative variants directly, with the positive variant taken as preferred. For both the within-positive and within-negative components, sequences are sorted by decreasing Δ and paired across a rank gap: for a pool of n sequences, the implementation sets

$$k = 1 + \lfloor \text{gap} (\lfloor n/2 \rfloor - 1) \rfloor$$

and, in successive blocks of $2k$ sequences, pairs each of the first k sequences with the corresponding sequence in the second k . The higher- Δ sequence is chosen. The configured value is $\text{gap}=0.5$.

The resulting preference relation therefore has the form

$$\Delta(x_{\text{chosen}}) > \Delta(x_{\text{rejected}}),$$

but the explicit separation into these four pair classes ensures that the training set contains both local comparisons within the positive and negative modes and larger-margin comparisons across the wild-type reference. In the final configuration, the pair mixture contains 10,000 within-positive pairs, 3,000 within-negative pairs, 2,000 wild-type anchor pairs, and 7,000 cross pairs.

The train, validation, and test sequence splits are generated before pair construction, and preference pairs are then built independently within each split. This prevents a sequence assigned to one split from appearing in a pair belonging to another split.

4 Methods

4.1 Fine-tuning methods

4.1.1 Domain-Adaptive Pretraining (DAP)

DAP continues masked-language-model training on the single-chain OAS corpus, with the aim of adapting ESM2 from broad protein sequence knowledge to the regularities of antibody sequences.

Training first uses conventional whole-chain masking to establish an antibody-domain reference model. We then compare two CDR-H3-oriented variants: a "CDR-focused" strategy that concentrates the prediction task within the IMGT-annotated CDR-H3 loop, and a hybrid strategy that combines CDR-focused examples with whole-chain examples. The latter retains a learning signal for the antibody framework while placing greater emphasis on the variable loop that is subsequently redesigned.

DAP was performed with masked-language-model training on OAS antibody sequences. Training began with standard whole-chain masking, where 15% of residues were masked. We then compared a CDR-focused strategy, masking 50% of positions within the CDR-H3 loop, with a hybrid strategy that applied

CDR-focused masking to 80% of sequences and whole-chain masking to the remaining 20%. The aim was to increase the learning signal on the CDR-H3 region while retaining information about the surrounding antibody framework.

Models are selected on the held-out validation split using masked-token recovery measured separately in CDR-H3 and framework positions. This balances improved CDR-H3 modelling against retention of general antibody-sequence knowledge. Selected runs are further refined with single-position masking around CDR-H3 and its local context, matching the conditional prediction setting used later for sequence scoring and generation. MLM loss and pseudo-perplexity are monitored as diagnostic measures; correlations with held-out DMS enrichment are evaluated only after model selection and therefore do not influence the DAP procedure.

4.1.2 Masked-DPO

Since Protein language models cannot assign a probability to a sequence, but can only assign a probability to a masked token given the context, the canonical DPO loss function cannot be applied directly to them. For this reason, rather than using the log-likelihood of the sequence, we used the pseudo-log-likelihood (PLL) of the sequence, which is defined as the sum of the log-probabilities of each token in the sequence given the context:

$$\text{PLL}_\theta(y) = \sum_{i=1}^L \log p_\theta(y_i \mid y_{-i})$$

where L is the length of the sequence, y_i is the i -th token in the sequence, and y_{-i} is the context of the sequence with the i -th token masked. Using the PLL, the DPO loss function can be rewritten as:

$$\mathcal{L}_{\text{DPO}} = -\mathbb{E}_{(y^w, y^l) \sim \mathcal{D}} [\log \sigma(\beta (\text{PLL}_\theta(y^w) - \text{PLL}_{\text{ref}}(y^w) - \text{PLL}_\theta(y^l) + \text{PLL}_{\text{ref}}(y^l)))] \quad (4)$$

To reduce the computational cost of evaluating the pseudo-log-likelihood, we restrict the score to the positions at which the chosen and rejected sequences differ. Let

$$M(y^w, y^l) = \{i : y_i^w \neq y_i^l\}$$

denote this set of positions, and define the restricted pseudo-log-likelihood score

$$S_\theta(y; M) = \sum_{i \in M} \log p_\theta(y_i \mid y_{\setminus i}).$$

Importantly, this restriction is not an exact cancellation of the contributions from shared residues: even when $y_i^w = y_i^l$, the corresponding conditional probabilities can differ because the surrounding sequence contexts are different. Instead, the restricted score provides a computationally cheaper approximation [4] that focuses the preference objective on the residues that distinguish the two sequences. For a pair-specific mask M , define the relative restricted score

$$r_\theta(y; M) = S_\theta(y; M) - S_{\text{ref}}(y; M).$$

As in the weighted DPO formulation described above, the winner and loser weights are computed from these relative scores:

$$w_M^{(i)} = \frac{\exp(\beta r_\theta(y^{(i)}; M))}{\sum_{j \in \{w, l\}} \exp(\beta r_\theta(y^{(j)}; M))}, \quad i \in \{w, l\}.$$

The weighted masked-DPO objective implemented in our experiments is therefore

$$\begin{aligned} \mathcal{L}_{\text{masked-DPO}} = & -\mathbb{E}_{(y^w, y^l) \sim \mathcal{D}} \left[w_M^w \log \sigma(\beta [r_\theta(y^w; M) - r_\theta(y^l; M)]) \right. \\ & \left. + w_M^l \log \sigma(\beta [r_\theta(y^l; M) - r_\theta(y^w; M)]) \right]. \end{aligned}$$

This is the same weighted winner–loser objective described in the Background, with full-sequence log-likelihood ratios replaced by the restricted pseudo-log-likelihood ratios. We refer to this approach as masked DPO, since the preference objective is evaluated only at the positions that differ between the chosen and rejected sequences, rather than over every position in the sequence.

4.1.3 LoRA-DPO

The LoRA-DPO approach uses the same loss function as the masked DPO, but it applies the LoRA technique to the attention layers of the model, in particular, the LoRA technique is applied to the query and value projection matrices of the attention layers, while keeping the rest of the model frozen.

4.2 Generation methods

4.2.1 Gibbs Sampling

CDR-H3 variants were generated with a Gibbs-style Markov chain using the masked-residue conditionals of the selected ESM2 checkpoint. The 24-residue loop was placed between the fixed left and right C05 sequence contexts. At each update, one CDR-H3 position i was selected uniformly at random and masked; the rest of the current sequence, including the other loop positions, remained visible to the model. Let $\ell_{i,a}$ denote the ESM2 logit for amino acid a at the masked position. The replacement residue was sampled from the temperature-scaled distribution over the 20 standard amino acids,

$$p_T(a \mid x_{/i}, c) = \frac{\exp(\ell_{i,a}/T)}{\sum_{b \in \mathcal{A}_{20}} \exp(\ell_{i,b}/T)},$$

where $x_{/i}$ denotes the current sequence with position i masked and c denotes the fixed framework context. The sampled residue replaced the previous residue, and the updated sequence became the state for the next iteration. Thus, updates were stochastic rather than greedy; temperature T controlled the concentration of the sampling distribution.

As a starting point for the Markov chain, two different initialization conditions were used:

1. **Distribution condition:** the initial sequence was drawn uniformly at random from a pool of 500 sequences sampled from the ED2 test set.
2. **Wild-type-start condition:** the initial sequence was the wild-type C05 CDR-H3.

Sampling was driven by the model’s conditional probabilities; experimental binding enrichment was not used to accept or reject proposals. Gibbs sampling turned out to continuously introduce new mutations as the chain progressed, and the resulting sequences were often far from the wild type; on one side, this allowed to explore a larger region of the sequence space, but on the other side it also resulted in a large number of sequences that had low PLL. In order to address this issue, we tried to cap the number of mutations that could be introduced in a sequence, but this did not result in a significant improvement in the PLL of the generated sequences.

4.2.2 Stochastic Beam Search

The stochastic beam search algorithm [14] operates on the CDR-H3 region while ESM2 receives the full variable heavy chain, with the CDR-H3 flanks held fixed. Only the 24 CDR-H3 positions are mutable, and substitutions are restricted to the 20 standard amino acids. For a sequence x , the implementation masks each mutable position i in turn and runs ESM2 on the resulting full sequence. The logits at the masked position are divided by temperature T , restricted to the standard amino acids, and normalized to give $q_{\theta,T}(a \mid x_{\setminus i})$. The sequence score is the CDR-H3 pseudo-log-likelihood

$$\text{PLL}_T(x) = \sum_{i \in \mathcal{M}} \log q_{\theta,T}(x_i \mid x_{\setminus i}),$$

where $\mathcal{M}$ is the set of mutable positions. This is a sum of masked-position conditional log probabilities, rather than a joint sequence log probability. At each iteration, every beam sequence is expanded to all single amino-acid substitutions at all mutable positions.

Now we need to compute a likelihood score for each new sequence: doing so would require a substantial computational effort, for this reason each candidate’s score is approximated by replacing the parent’s conditional log-probability term at the mutated position. Candidates are then deduplicated and, when a candidate arises from more than one parent, retains its highest assigned score. The selected candidates are then rescored from their own masked-position conditionals during the next iteration. To select the next beam, the algorithm adds independent standard Gumbel noise to each candidate score and retains the B candidates with the largest perturbed scores. This Gumbel-top- B step gives a stochastic, weighted

selection among the current candidate pool. The unperturbed scores are retained for the next expansion. Parents are not automatically carried forward, and each step’s candidates differ from a parent by one substitution. A substitution can reverse an earlier change, so the number of steps bounds mutation moves rather than the final Hamming distance from the seed. Temperature affects the amino-acid conditionals and the candidate scores relative to the fixed-scale Gumbel noise. The starting sequence is configurable: it can be the wild type, a randomly selected DMS sequence, or a high-scoring DMS sequence. In `top_dms` mode, the code runs an independent search from each selected seed. The `snapshot_every` setting controls which beam states are written to the output; it does not change the search. The search is summarized in Algorithm 1. `ScoreAndConditionals` masks each mutable position in turn, computes its temperature-scaled conditional distribution, and sums the log probabilities assigned to the sequence’s existing residues.

Algorithm 1 Stochastic beam search for CDR-H3 optimization

Require: Seed sequence x_0 , mutable positions $\mathcal{M}$, amino-acid alphabet $\mathcal{A}$, beam width B , maximum number of steps N , temperature T

Ensure: Beam states $\mathcal{B}_0, \dots, \mathcal{B}_N$

```

1:  $\mathcal{B}_0 \leftarrow \{x_0\}$ 
2: for  $t \leftarrow 1$  to  $N$  do
3:    $\mathcal{C} \leftarrow$  empty map from sequences to scores
4:   for all  $x \in \mathcal{B}_{t-1}$  do
5:      $(\{q_i(\cdot | x)\}_{i \in \mathcal{M}}, \ell_T(x)) \leftarrow \text{SCOREANDCONDITIONALS}(x, T)$ 
6:     for all  $i \in \mathcal{M}$  do
7:       for all  $a \in \mathcal{A} \setminus \{x_i\}$  do
8:          $x' \leftarrow x$  with residue  $i$  replaced by  $a$ 
9:          $\ell' \leftarrow \ell_T(x) - \log q_i(x_i | x) + \log q_i(a | x)$ 
10:         $\mathcal{C}[x'] \leftarrow \max(\mathcal{C}[x'], \ell')$ 
11:      end for
12:    end for
13:  end for
14:  if  $\mathcal{C}$  is empty then
15:    break
16:  end if
17:  for all  $x' \in \text{keys}(\mathcal{C})$  do
18:     $g_{x'} \sim \text{Gumbel}(0, 1)$ 
19:     $z_{x'} \leftarrow \mathcal{C}[x'] + g_{x'}$ 
20:  end for
21:   $\mathcal{B}_t \leftarrow$  sequences for the top  $\min(B, |\mathcal{C}|)$  values of  $z_{x'}$ 
22: end for
23: return  $\mathcal{B}_0, \dots, \mathcal{B}_N$ 

```

In the score update, $q_i(\cdot | x)$ is calculated from the masked parent sequence. The code recalculates these conditionals when a selected candidate becomes a parent in the next iteration.

4.2.3 Random library-mutant sampler

The random library-mutant sampler is used as a baseline for comparison with the other generation methods, it is the simplest method for generating new sequences and relies on no information about the data distribution. We start from the C05 wild-type sequence and look at the ED2 training data, keeping only the sequences that score above the WT binding enrichment. For each of the 24 loop positions we collect the set of amino acids that appear there (the WT residue is always included). A position is called mutable if more than one residue was seen at it, meaning the data gives us real alternatives to try.

To make one new sequence, we draw the number of positions to change, k , by uniform random sampling from $\{1, \dots, \text{trust radius}\}$, and we then pick which k mutable positions to change uniformly at random. We change each chosen position to one of the other residues seen there, again drawn uniformly at random over that position’s observed set (never the residue already there). The trust radius is the max number of changes from WT we can make at once; we set it to 11, the same as the largest number of steps the DPO beam search takes.

4.2.4 PSSM sampler

The PSSM sampler is a simple generative model that uses the empirical amino-acid frequencies at each position in the training data to generate new sequences. It is used as a baseline for comparison with the other generation methods, it’s slightly more sophisticated than the random library-mutant sampler, as it uses the empirical amino-acid frequencies from the dataset to generate new sequences. Using the same above-WT filter on the ED2 training split, we count how often each of the 20 amino acids appears at each of the 24 loop positions and normalise the counts into a probability. This gives a 24×20 table where each row is a probability distribution over amino acids for one position. To generate a sequence, we independently sample at every position using its row of the table.

A temperature parameter T lets us make these distributions more or less peaked before sampling: we raise each column’s probabilities to the power $1/T$ and normalise again, so $T < 1$ pushes sampling toward the most common residue, while $T > 1$ makes it more even. We use $T = 0.9$, which keeps the generated sequences close to the real data while still giving enough variety.

5 Experiments and Results

5.1 Evaluation Metrics

Binding-score ranking. The primary measure of predictive performance is Spearman’s rank correlation between a model sequence score $z(x)$, typically the CDR-H3 pseudo-log-likelihood (PLL), and the measured binding enrichment $s(x)$:

$$\rho_S = \text{corr}(\text{rank}(z(x)), \text{rank}(s(x))).$$

A higher positive value indicates better agreement between model ranking and experimental enrichment. Because enrichment is bimodal, correlation is also evaluated separately for sequences with $\Delta > 0$ and $\Delta \leq 0$. The area under the receiver operating characteristic curve (AUROC) measures discrimination between these two groups, using $\Delta > 0$ as the positive class. Spearman p -values and the sample counts in each group are recorded as supporting statistics.

Likelihood and perplexity. Language-model fit is measured by masked-language-model loss and pseudo-perplexity during DAP, and by PLL-based pseudo-perplexity (PPPL) during DPO and sequence evaluation. For a set of sequences $\mathcal{D}$, corpus-level pseudo-perplexity is

$$\text{PPPL}(\mathcal{D}) = \exp\left(-\frac{\sum_{x \in \mathcal{D}} \text{PLL}(x)}{\sum_{x \in \mathcal{D}} L_x}\right),$$

where L_x is the number of scored residues. DPO pseudo-perplexity scores CDR-H3 residues while conditioning on the surrounding sequence. Lower pseudo-perplexity indicates that the model assigns greater probability to the evaluated sequences. pseudo-perplexity is reported for preferred sequences in the pairwise validation and test sets, and separately for positive-enrichment, negative-enrichment, and wild-type reference sequences. These subset values show whether a change in average likelihood is concentrated in a particular enrichment group.

Preference-learning metrics. For each preference pair (x_c, x_r) , the implicit reward is the policy’s PLL relative to the reference model, scaled by β :

$$r_\theta(x) = \beta [\text{PLL}_\theta(x) - \text{PLL}_{\text{ref}}(x)].$$

Reward accuracy is the fraction of pairs for which $r_\theta(x_c) > r_\theta(x_r)$; values closer to one indicate more correctly ordered pairs. Reward margin is the mean difference $r_\theta(x_c) - r_\theta(x_r)$, for which larger positive values indicate stronger separation. Both are calculated on the sequence positions that differ within each pair. The DPO objective loss is evaluated on training and validation pairs, with lower values indicating a better fit to the preference objective. The reported implicit-KL diagnostic is the mean policy-minus-reference PLL over the chosen and rejected sequences; it tracks policy drift from the reference and is not a direct estimate of a normalized sequence-distribution KL divergence.

Validation and test protocol. DPO pair loss and reward metrics are computed on the train, validation, and held-out pair partitions of ED2. Separately, binding-score correlations and AUROC are evaluated on held-out DMS data; the DPO test Spearman analysis uses the configured external test set and its two-mutation variants. Checkpoint selection for DPO prioritizes the highest validation Spearman correlation, falling back to the lowest validation DPO loss when that correlation is unavailable. DAP monitors validation MLM loss and pseudo-perplexity, and also records Spearman-based checkpoints. Across methods, external test metrics are computed after selecting the checkpoint using validation data.

Sequence-generation diagnostics. Generated sequences are assessed using CDR-H3 PLL, mutation count relative to wild type, and sequence diversity. The fraction above wild type is the proportion whose PLL exceeds the wild-type PLL; it measures model likelihood and is not an experimental binding-success rate. Diversity is summarized by mean pairwise Hamming distance and position-wise amino-acid entropy,

$$H_i = - \sum_{a \in \mathcal{A}} p_i(a) \log_2 p_i(a),$$

where $p_i(a)$ is the frequency of amino acid a at position i . Distributional agreement with DMS sequences is measured by the Jensen-Shannon divergence between their position-wise amino-acid frequency distributions; lower values indicate closer marginal agreement. For enrichment recovery, generated sequences are ranked by PLL and the mean measured enrichment of their nearest DMS neighbors is reported for the top- K sequences. This is a nearest-neighbor estimate, not a measured enrichment for the generated sequences themselves. Novelty is the fraction of generated sequences absent by exact sequence match from the indexed DMS datasets.

5.2 Domain-Adaptive Pretraining Analysis

Given the extremely variable nature of the CDR-H3 loop, DAP is not guaranteed to improve the model’s ability to predict binding enrichment, hence it is important to evaluate its effect on the model’s performance. In this experiment we evaluate the effect of DAP by comparing the performance of the base ESM2 model with that of the DAP-adapted models; we use the 650M parameter model for this experiment.

The performance of the models is evaluated on a held-out portion of the ED2 and on the ED5 and ED811 datasets, which were not used for training. The metrics used are the Spearman correlation between the model’s pseudo-log-likelihood and the measured binding enrichment (Figure 3), and the pseudo-perplexity of the model on the CDR-H3 loop of the C05 wild type and the held-out ED2 variants (Figure 4).

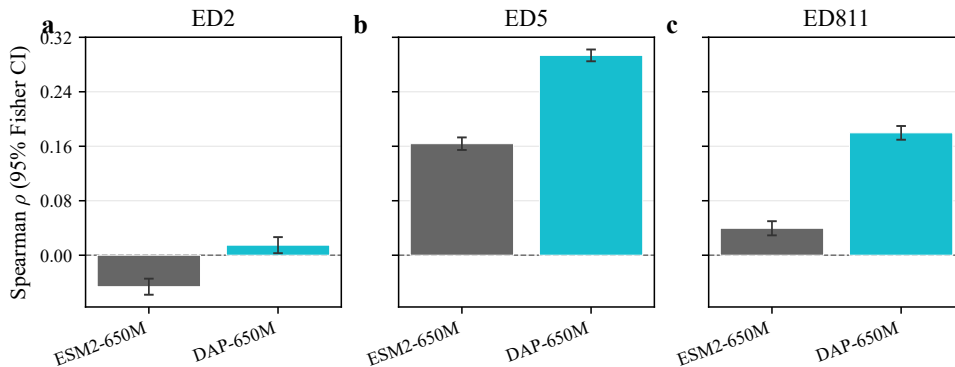


Figure 3: Held-out Spearman correlation after domain-adaptive pretraining (DAP); error bars are approximate 95% Fisher-transformation confidence intervals.

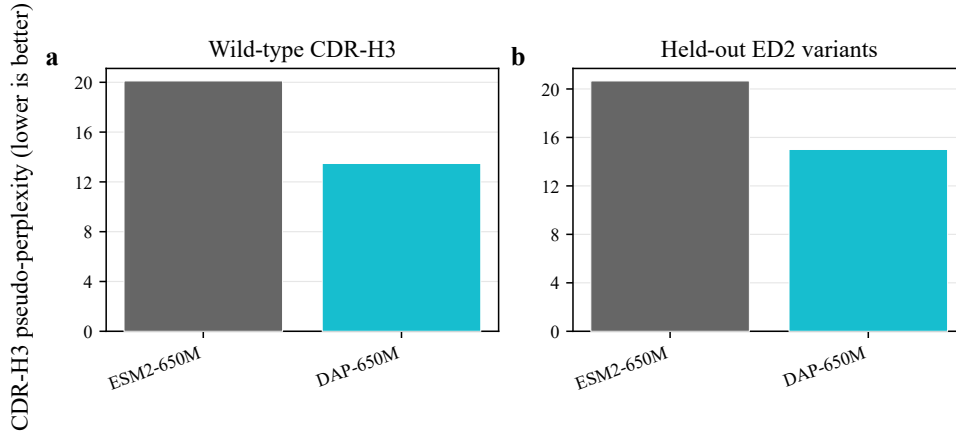


Figure 4: Pseudo-perplexity of the model on the C05 wild type and held-out ED2 CDR-H3 loop variants after domain-adaptive pretraining (DAP).

At first glance, the results are quite promising; however one important consideration must be taken into account when interpreting these results. The overall Spearman correlation on ED5 and ED811 is partly driven by separation between the two enrichment modes, and therefore does not purely measure fine-grained ranking within the binding and non-binding groups.

In the light of this consideration, we can consider the Spearman correlation computed on the held-out portion of the ED2 dataset as a more reliable metric to evaluate the effect of DAP on the model’s performance. As shown in Figure 3, the Spearman correlation on the held-out portion of the ED2 dataset is improved by DAP, however its value remains close to zero and therefore provides only weak evidence of useful fine-grained ranking. On both the ED5 and ED811 datasets, the Spearman correlation is improved by DAP, meaning that the model is better able to separate the two major enrichment modes, even though the overall correlation is partly driven by separation between the two enrichment modes, this is still a positive result, since in the filtered OAS corpus used for DAP there is no direct information about the binding enrichment of the sequences.

The metric showing the largest change after DAP is the pseudo-perplexity of the model on the CDR-H3 loop of the sequences in the datasets. As shown in Figure 4, the pseudo-perplexity of the model on the CDR-H3 loop is substantially reduced after DAP, even though the filtered OAS corpus used for DAP is not particularly enriched in sequences that are similar to the C05 wild type, this is consistent with DAP improving antibody-specific sequence modeling.

5.3 DPO analysis

In this experiment we investigate the effect of DPO on the model’s performance, and whether a DAP-adapted model is more suitable for DPO than a base model. We evaluate the performance of the DPO models on a held-out portion of the ED2 dataset, and on the ED5 and ED811 datasets, which were not used for training.

The first goal is to determine the effectiveness of DPO training. This was done by evaluating the reward accuracy, the reward margin, and the DPO loss on the held-out portion of the ED2 dataset, as shown in Figure 5. All DPO variants achieve held-out reward accuracies substantially above the 0.5 random-ordering baseline and positive reward margins, indicating that preference training successfully transfers to unseen ED2 preference pairs. Full-parameter DPO and DAP+DPO obtain slightly stronger margins than LoRA-DPO, while reward accuracy is relatively similar across models. The held-out DPO loss is also lower for the full-parameter models than for LoRA-DPO, although this metric is less directly interpretable than reward accuracy and margin.

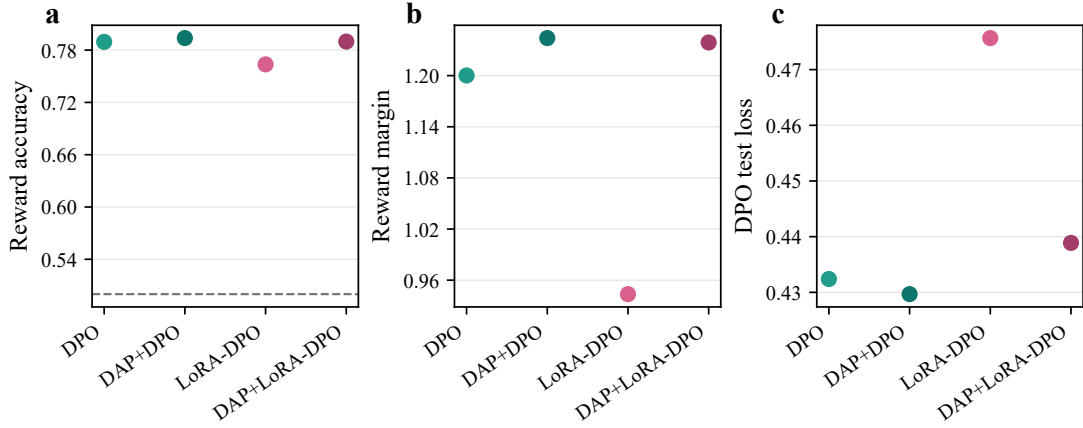


Figure 5: Held-out preference metrics for DPO.

In Figure 6 we show the Spearman correlation between the model’s pseudo-log-likelihood and the measured binding enrichment for all the DPO models. What immediately stands out is that all DPO models have an appreciably higher Spearman correlation than the base and DAP-adapted models, meaning that DPO training is effective in improving the model’s ability to distinguish between sequences with different binding enrichment scores. This is particularly true when evaluating the Spearman on the held-out portion of the ED2 dataset, and on the ED5 dataset, on the other hand, on the ED811 dataset, the models that underwent DAP before DPO training have a lower Spearman correlation than the models without DAP, meaning that DAP is not always beneficial for DPO training, and that its effect on DPO training is highly dependent on the dataset used for evaluation.

The AUROC metric, shown in Figure 7, confirms the results obtained with the Spearman correlation; all DPO models have an appreciably higher AUROC than the base and DAP-adapted models, meaning that DPO training is effective in improving the model’s ability to distinguish variants with enrichment above WT from variants at or below WT. It is also interesting to notice that the dataset on which the DPO models were trained, the ED2 dataset, is the one on which the DPO models have the highest AUROC, while on the other two datasets, on which some models showcased a higher Spearman correlation, the AUROC is lower. A summary of the performance of all models on the held-out evaluation sets is shown in Table 1.

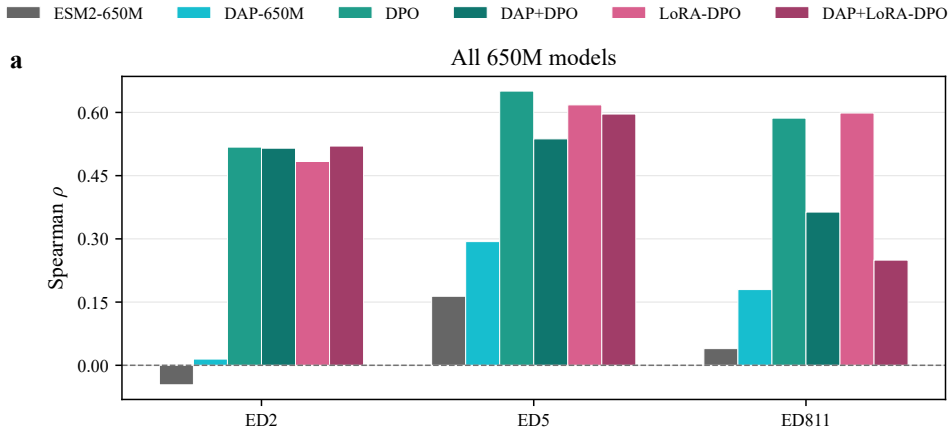


Figure 6: Spearman correlation between model PLL and binding enrichment on held-out test sets for all DPO models.

Model	Reward acc.	Reward margin	WT CDR PPPL	ED2 ρ	ED5 ρ	ED8-11 ρ
ESM2-650M	—	—	20.11	-0.046	0.164	0.040
DAP-650M	—	—	13.48	0.015	0.293	0.180
DPO	0.789	1.200	2.33	0.518	0.651	0.586
DAP+DPO	0.794	1.244	2.05	0.515	0.537	0.364
LoRA-DPO	0.764	0.944	3.07	0.484	0.618	0.599
DAP+LoRA-DPO	0.790	1.239	2.18	0.520	0.596	0.250

Table 1: Model summary on held-out evaluation sets.

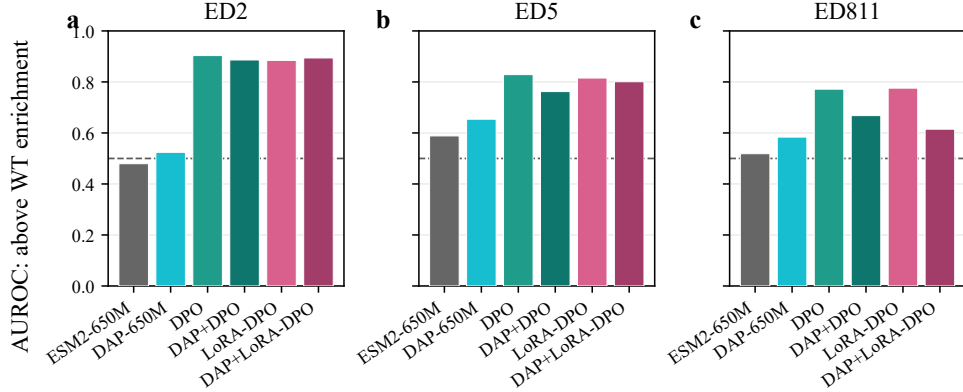


Figure 7: AUROC on held-out test sets comparison between DPO models differentiated by base model.

5.4 Generated sequences analysis

In this section we analyze the sequences generated by the different models, comparing the different models and the different generation methods, and evaluating the quality and diversity of the generated sequences. We use two different baseline generation methods, a random generation method, and a PSSM-based generation method.

In the comparison we generate 5000 sequences for each model and generation method, the beam search has a beam size of $B = 500$, and a maximum number of steps of 11. The random generation method has a trust radius of 11, and the PSSM-based generation method has a temperature of $T = 0.9$.

5.4.1 Exploration and novelty

First we want to investigate how the generators explore the sequence space, and how novel the generated sequences are with respect to the training data. In Figure 8 we evaluate the generated sequences by looking at the mutation distance from the wild type, the plot shows that PSSM remains very close to WT (median distance 1), while DPO stochastic beam reaches substantially farther (median 6); random reaches similarly far but without model-guided selection.

Generator	generated	unique (dedup)	unique + novel
Random ($R=11$)	5000	4840 (96.8%)	4271 (88.2%)
PSSM ($T=0.9$)	5000	2647 (52.9%)	1279 (48.3%)
DPO 650M (beam search)	5000	3934 (78.7%)	3451 (87.7%)

Table 2: Library sizes and the effect of each filter. Each generator emits 5000 sequences; “unique” keeps one copy of each distinct CDR-H3 (deduplication) and “unique + novel” additionally drops sequences already present in the generators’ training data.

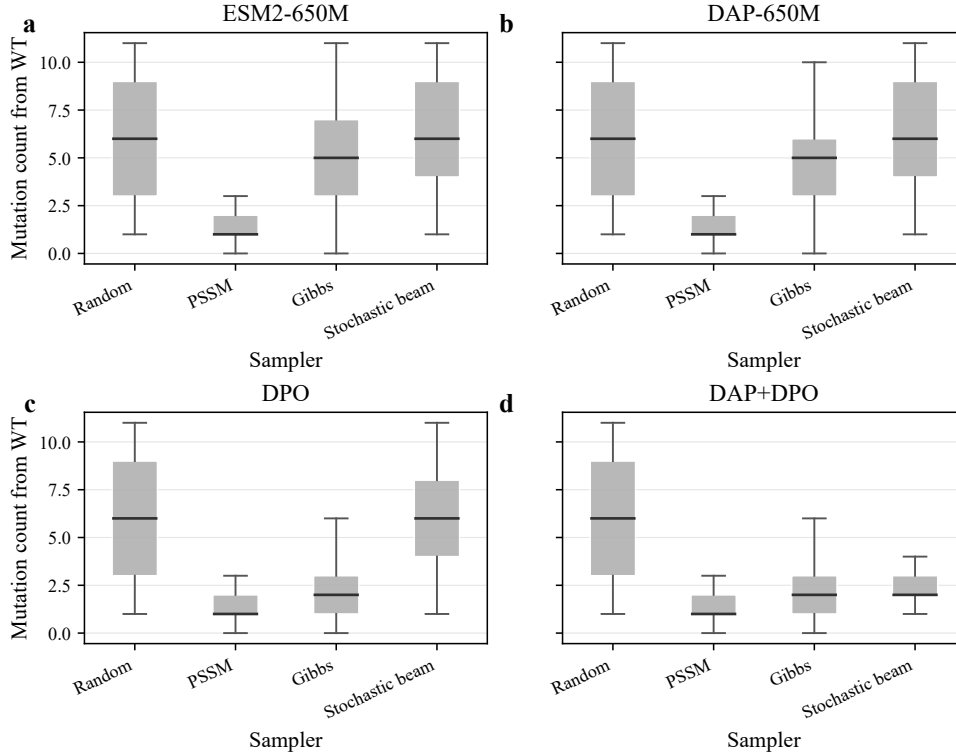


Figure 8: Mutation distance of generated sequences by model.

Second, we want to see how novel the generated sequences are with respect to the training data, i.e. if the generated sequences are already present in the training data or if they are new sequences. Table 2 compares the number of generated sequences, the number of unique sequences after deduplication, and the number of unique sequences that are also absent from the generators’ training data for the random, PSSM, and DPO+stochastic beam generators. The random generator produces a large number of unique sequences, of which about 88% are novel, the same portion of novel sequences is produced by the DPO generator, while the PSSM generator produces a much smaller number of unique and novel sequences. However the diversity of the sequences generated by the random sampler arises from the fact that it is not guided by any model, on the other hand, the DPO generator is guided by the model and produces a large number of unique and novel sequences with several mutations away from the wild type, while achieving a large novelty with respect to the training data.

Preliminary experiments compared Gibbs sampling and stochastic beam search across model checkpoints. The more controlled baseline comparison below focuses on DPO-650M with stochastic beam search, which was selected as the main generation configuration.

5.4.2 Quality of generated candidates

The first question is whether the generated sequences are assigned high binding scores by models that were not used during generation. This is important because evaluating the generated sequences only with the same DPO model used to guide the search would give a partially circular evaluation.

For this reason, we evaluate the generated libraries with two independent in-silico scorers. The first one is the PGLD scorer, an ESM2-based regression model trained to predict binding enrichment. The second one is an ensemble-based scorer that also provides an uncertainty estimate. For both scorers, the wild-type prediction is used as a reference value.

Since the final goal of the generation procedure is to provide a pool of candidates for Bayesian optimization or experimental testing, we are mainly interested in the highest-scoring part of each generated library rather than in the average score over the full library. We therefore rank the generated sequences according to the external scorer and compute the mean score of the top- N sequences for increasing values of N .

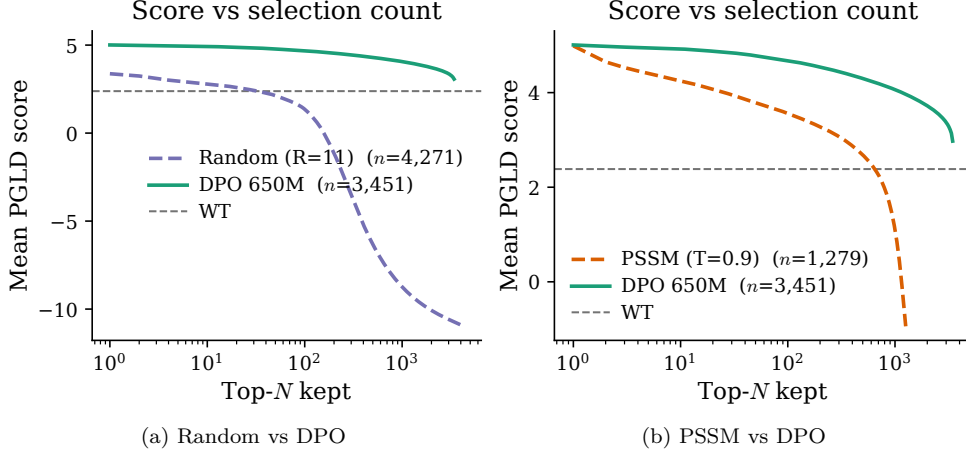


Figure 9: **Scott's PGLD scorer**. Mean predicted PGLD score of the top- N sequences of each library (sorted by PGLD score); the left end ($N = 1$) is the single best sequence and each curve ends at its own library size. Dashed line = WT prediction (2.384). Subset: unique + novel designs, no mutation cap ($n = 4271$ Random, 1279 PSSM, 3451 DPO). DPO is above both baselines at every selection count and stays above WT across almost the whole range.

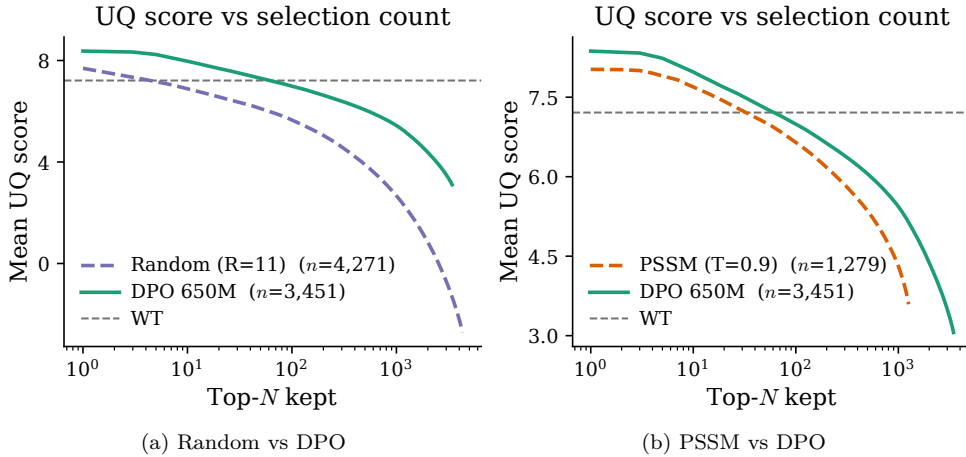


Figure 10: **Lorenz UQ scorer**. Same top- N view under the ensemble scorer (its own scale). Subset: unique + novel designs, no mutation cap ($n = 4271$ Random, 1279 PSSM, 3451 DPO). Dashed line = WT prediction (7.211). DPO is at or above both baselines across the whole range, and its top sequences stay above WT longer.

As shown in Figure 9, Figure 10, the highest-ranked DPO-generated sequences receive higher predicted scores than the corresponding sequences generated by the random and PSSM baselines. Under the PGLD scorer, the difference remains visible as the number of selected sequences is increased. A similar trend is observed with the ensemble scorer, where the highest-ranked DPO candidates are also assigned higher scores than the baseline-generated candidates.

A direct comparison of the complete generated libraries is complicated by the different mutation-distance distributions produced by the three methods. In particular, the PSSM remains much closer to the wild

Model	n	mean	median	unc.	frac > WT
Random ($R=11$)	1475	0.54	0.98	9.78	0.1%
PSSM ($T=0.9$)	1263	3.51	3.73	7.13	1.0%
DPO 650M	1347	4.61	4.79	4.83	1.9%

Table 3: External ensemble-scorer evaluation restricted to unique and novel generated sequences within five mutations of the C05 wild type. "unc." denotes mean total predictive standard deviation from the ensemble scorer; lower values indicate lower predictive uncertainty.

type, while the DPO and random generators explore sequences with a larger number of mutations. Since the uncertainty of the ensemble scorer increases for sequences further away from the wild type, we also compare the three generators after restricting all libraries to sequences with at most five mutations from the wild type.

Within this matched-distance regime, the DPO-generated sequences obtain a higher mean and median predicted score than the two baselines. They also show a lower mean predictive uncertainty. This suggests that the difference between the generators cannot be explained only by the different distances from the wild type.

These results should still be interpreted as in-silico evaluation. Neither external scorer provides direct experimental evidence that the generated sequences have improved binding. Their main role here is to provide an evaluation signal that is independent from the model used during generation.

5.4.3 Quality–diversity trade-off

The three generation methods have different mechanisms for controlling the amount of exploration. For the random sampler, the trust radius controls the maximum number of mutations from the wild type. For the PSSM sampler, the sampling temperature controls how strongly the generated residues follow the empirical amino-acid frequencies. Reducing the trust radius or the temperature generally increases similarity to the wild type, but also reduces the diversity of the generated library.

To study whether the higher scores of the DPO-generated candidates can be reproduced by simply making the baseline generators more conservative, we compare predicted quality and sequence diversity over a sweep of these baseline hyperparameters. Diversity is measured using the mean pairwise Hamming distance between generated CDR-H3 sequences, while sequence quality is evaluated using the independent PGLD scorer.

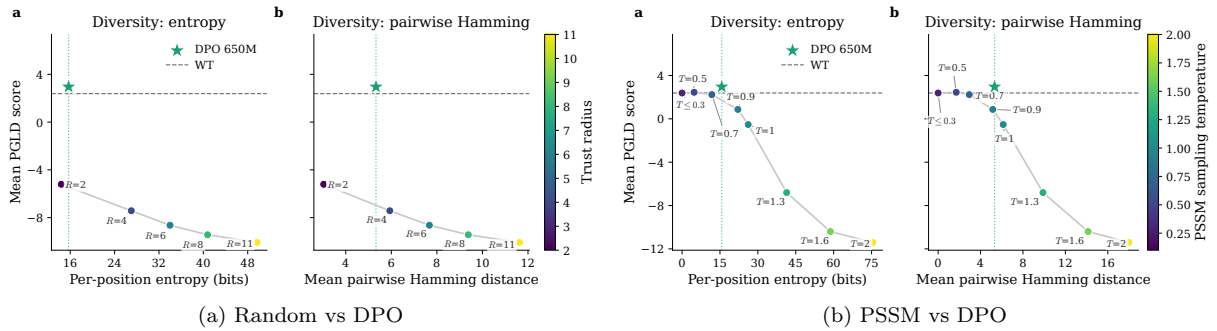


Figure 11: Quality (duplicate-weighted mean predicted PGLD score) versus diversity (total per-position entropy, left version; mean pairwise Hamming, right version). Subset: the novel sequences of each library with duplicates kept. Each baseline forms a curve (one point per temperature/trust-radius); DPO is a single star. The DPO operating point lies above the baseline quality-diversity curves, achieving higher predicted quality at comparable diversity.

Figure 11 shows that both baseline generators exhibit a clear trade-off between predicted score and diversity. Their predicted quality improves when the generated sequences become more similar to the wild type, while increasing the amount of exploration decreases the average predicted score.

The DPO stochastic-beam generator reaches a different region of this trade-off. Its generated sequences maintain comparatively high predicted scores while still producing a diverse library. Therefore, the

improvement of the DPO generator cannot be reproduced only by reducing the mutation distance or the diversity of the baseline samplers.

This behaviour is consistent with the fact that the DPO model evaluates mutations in the context of the complete sequence. In contrast, the PSSM samples residues independently at each position and the random baseline selects mutations without using a sequence-level score. The result does not prove that the DPO model has learned the true epistatic structure of the binding landscape, but it suggests that sequence-context information is useful for directing the generation process.

5.4.4 Mutation patterns in generated sequences

In addition to the overall mutation distance, we also analyse where the different generators introduce mutations along the CDR-H3 loop. This allows us to verify whether the DPO model distributes changes uniformly across the loop or whether it concentrates them on a smaller number of positions.

For each generated library, we compute the fraction of sequences containing a non-wild-type residue at each of the 24 CDR-H3 positions.

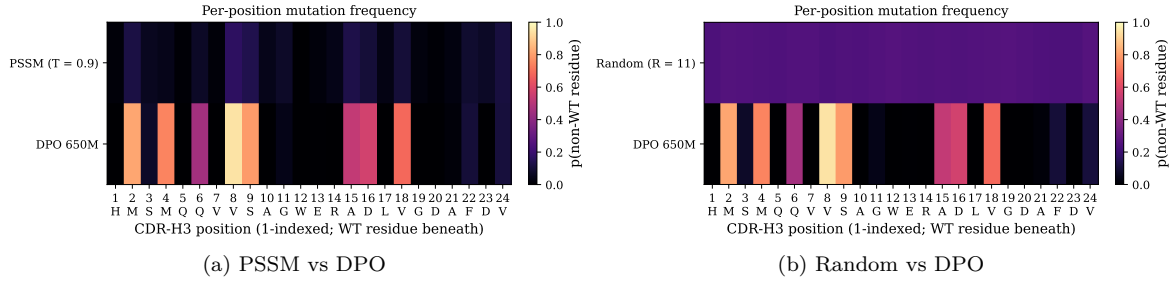


Figure 12: Per-position mutation frequency: the chance that a generated sequence has a non-WT residue at each position. DPO concentrates its mutations on a smaller subset of positions, while the baselines distribute changes more broadly.

The random sampler distributes mutations relatively broadly across the mutable positions, as expected from its construction. The PSSM remains strongly concentrated around the wild-type sequence, with most positions showing a low mutation frequency. In contrast, the DPO stochastic-beam generator concentrates a larger fraction of its mutations on a smaller subset of positions.

This indicates that the DPO model has learned a non-uniform preference over CDR-H3 positions from the DMS preference data. However, these mutation frequencies should not be interpreted as direct evidence that the corresponding positions are causally responsible for binding. They only show which mutations are preferred by the trained model during generation.

5.4.5 Model disagreement and generation failure modes

The previous analyses use independent scorers to evaluate the generated sequences. We now compare these scores directly with the PLL of the DPO model used during generation. This comparison is important because stochastic beam search explicitly selects sequences according to the DPO model score, and therefore can move towards regions of sequence space where this score is high.

For the DPO-generated library, we compare the DPO CDR-H3 PLL with the predictions of the PGLD scorer and the ensemble scorer. Each sequence is also coloured according to its Hamming distance from the wild type.

Figure 13 shows a strong disagreement between the DPO model score and the independent scorers on sequences produced by the DPO-guided search. In particular, sequences with high DPO PLL are not necessarily assigned high scores by the external predictors. The disagreement becomes more visible for sequences further away from the wild type.

This result is different from the held-out DMS evaluation, where the DPO PLL showed a clear positive correlation with measured binding enrichment. It suggests that the behaviour of the model changes when it is used not only to score existing sequences, but also to directly guide the search towards sequences that maximize its own score.

One possible explanation is a search-induced distribution shift. The stochastic beam search can move towards regions of sequence space that were poorly represented in the DMS training data, where the learned relation between model likelihood and binding enrichment may no longer hold. The search may

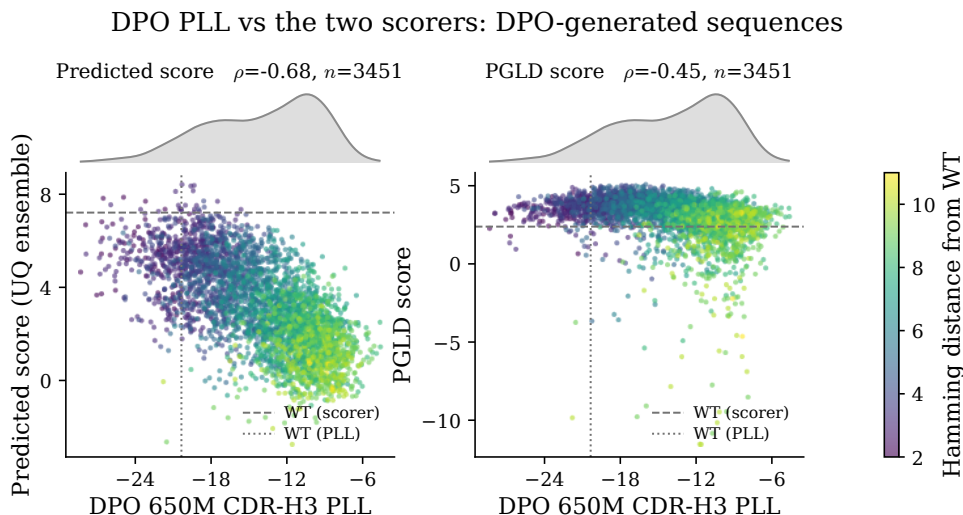


Figure 13: DPO model pseudo-log-likelihood versus the predictions of two independent in-silico scorers on DPO-generated sequences. Points are coloured by Hamming distance from the C05 wild type.

also exploit model-specific features that increase PLL without producing the corresponding improvement according to independent predictors.

The external scorers are themselves imperfect models and therefore this disagreement is not sufficient to conclude that the DPO generator produces poor binders. However, it shows that the DPO PLL should not be interpreted as a direct estimate of binding affinity outside the evaluation distribution.

For this reason, the generative model is better interpreted as a proposal mechanism rather than as a complete sequence-design solution. The generated candidates should be filtered using independent predictors and uncertainty estimates before being passed to Bayesian optimization or experimental validation.

6 Conclusion

In this project we investigated whether a pretrained Protein Language Model can be adapted into a useful generative model for proposing new C05 CDR-H3 variants. We considered two complementary adaptation strategies: domain-adaptive pretraining (DAP) on antibody sequences from the Observed Antibody Space, and Direct Preference Optimization using experimentally measured C05 binding-enrichment data.

DAP substantially reduced the pseudo-perplexity of the model on the C05 CDR-H3 sequence and on held-out C05 variants, showing that continued training on antibody sequences improves antibody-specific sequence modeling. Its effect on binding-enrichment ranking was more limited. The Spearman correlation on held-out ED2 variants remained weak, while the larger improvements observed on ED5 and ED811 were partly driven by the strongly bimodal structure of these datasets.

DPO had a much stronger effect on binding-enrichment ranking. All DPO variants achieved held-out reward accuracies clearly above random ordering and positive reward margins. More importantly, DPO substantially increased the correlation between model pseudo-log-likelihood and measured binding enrichment on held-out DMS datasets. The results also showed that DAP before DPO was not consistently beneficial across evaluation datasets, while full-parameter and LoRA-based DPO produced broadly similar preference-learning behaviour.

We then used the trained models for sequence generation with Gibbs sampling and stochastic beam search. For the main comparison, DPO-guided stochastic beam search was evaluated against a random mutation baseline and a PSSM sampler. The DPO generator produced a large number of unique and training-novel variants while exploring several mutations away from the wild type. Compared with the two baselines, its highest-ranked candidates obtained better predictions from two independent in-silico scorers. This advantage was still visible after restricting the comparison to sequences within five mutations of the wild type, and the DPO generator also reached a more favourable quality-diversity trade-off than the baseline samplers.

At the same time, the generation experiments exposed an important limitation. On DPO-generated sequences, the DPO model’s own pseudo-log-likelihood showed strong disagreement with the independent

scorers, especially for sequences further away from the wild type. This suggests that directly optimizing the model score can move the search into regions that differ from the distribution on which the relation between model likelihood and binding enrichment was learned. The DPO score should therefore not be interpreted as a direct estimate of binding affinity for arbitrarily generated sequences.

Overall, the results show that preference optimization can turn a pretrained Protein Language Model into a useful proposal mechanism for antibody sequence design. The model is able to learn the experimental preference signal and use it to guide generation toward regions that are more promising than those reached by simple random or position-independent baselines. However, the generated candidates still require independent filtering and experimental validation. A natural continuation of this work is therefore to use the generated libraries as proposal sets for Bayesian optimization, where model predictions, uncertainty estimates, and new experimental measurements can be combined iteratively to refine the search for improved C05 variants.

References

- [1] Ethan C. Alley et al. “Unified rational protein engineering with sequence-based deep representation learning”. In: *Nature Methods* 16 (2019), pp. 1315–1322. DOI: [10.1038/s41592-019-0598-1](https://doi.org/10.1038/s41592-019-0598-1).
- [2] Tobias H. Olsen, Iain H. Moal, and Charlotte M. Deane. “AbLang: an antibody language model for completing antibody sequences”. In: *Bioinformatics Advances* 2.1 (2022), vbac046. DOI: [10.1093/bioadv/vbac046](https://doi.org/10.1093/bioadv/vbac046).
- [3] Talal Widatalla et al. “Aligning protein-generative models to experimental fitness with ProteinDPO”. In: *Nature Methods* 23.9 (Sept. 2026), pp. 1805–1813. ISSN: 1548-7105. DOI: [10.1038/s41592-026-03137-3](https://doi.org/10.1038/s41592-026-03137-3). URL: <https://doi.org/10.1038/s41592-026-03137-3>.
- [4] Constance Ferragu et al. *g-DPO: Scalable Preference Optimization for Protein Language Models*. 2025. arXiv: [2510.19474](https://arxiv.org/abs/2510.19474) [cs.LG]. URL: <https://arxiv.org/abs/2510.19474>.
- [5] Long Ouyang et al. “Training language models to follow instructions with human feedback”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022.
- [6] Zeming Lin et al. “Evolutionary-scale prediction of atomic-level protein structure with a language model”. In: *Science* 379.6637 (2023), pp. 1123–1130. DOI: [10.1126/science.ade2574](https://doi.org/10.1126/science.ade2574). eprint: <https://www.science.org/doi/pdf/10.1126/science.ade2574>. URL: <https://www.science.org/doi/abs/10.1126/science.ade2574>.
- [7] Suchin Gururangan et al. “Don’t Stop Pretraining: Adapt Language Models to Domains and Tasks”. In: *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*. Online: Association for Computational Linguistics, July 2020, pp. 8342–8360. DOI: [10.18653/v1/2020.acl-main.740](https://doi.org/10.18653/v1/2020.acl-main.740). URL: <https://aclanthology.org/2020.acl-main.740/>.
- [8] Rafael Rafailov et al. *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. 2024. arXiv: [2305.18290](https://arxiv.org/abs/2305.18290) [cs.LG]. URL: <https://arxiv.org/abs/2305.18290>.
- [9] Andreas Krause and Jonas Hübner. *Probabilistic Artificial Intelligence*. 2025. arXiv: [2502.05244](https://arxiv.org/abs/2502.05244) [cs.AI]. URL: <https://arxiv.org/abs/2502.05244>.
- [10] Edward J. Hu et al. “LoRA: Low-Rank Adaptation of Large Language Models”. In: *CoRR* abs/2106.09685 (2021). arXiv: [2106.09685](https://arxiv.org/abs/2106.09685). URL: <https://arxiv.org/abs/2106.09685>.
- [11] James R. R. Whittle et al. “Broadly neutralizing human antibody that recognizes the receptor-binding pocket of influenza virus hemagglutinin”. In: *Proceedings of the National Academy of Sciences* 108.34 (Aug. 2011), pp. 14216–14221. ISSN: 0027-8424. DOI: [10.1073/pnas.1111497108](https://doi.org/10.1073/pnas.1111497108). URL: <https://doi.org/10.1073/pnas.1111497108>.
- [12] Douglas M. Fowler and Stanley Fields. “Deep mutational scanning: a new style of protein science”. In: *Nature Methods* 11.8 (Aug. 2014), pp. 801–807. ISSN: 1548-7105. DOI: [10.1038/nmeth.3027](https://doi.org/10.1038/nmeth.3027). URL: <https://doi.org/10.1038/nmeth.3027>.
- [13] Tobias H. Olsen, Fergus Boyles, and Charlotte M. Deane. “Observed Antibody Space: A diverse database of cleaned, annotated, and translated unpaired and paired antibody sequences”. In: *Protein Science* 31.1 (Jan. 2022), pp. 141–146. ISSN: 0961-8368. DOI: [10.1002/pro.4205](https://doi.org/10.1002/pro.4205). URL: <https://doi.org/10.1002/pro.4205>.
- [14] Wouter Kool, Herke van Hoof, and Max Welling. “Stochastic Beams and Where to Find Them: The Gumbel-Top-k Trick for Sampling Sequences Without Replacement”. In: *CoRR* abs/1903.06059 (2019). arXiv: [1903.06059](https://arxiv.org/abs/1903.06059). URL: <https://arxiv.org/abs/1903.06059>.